

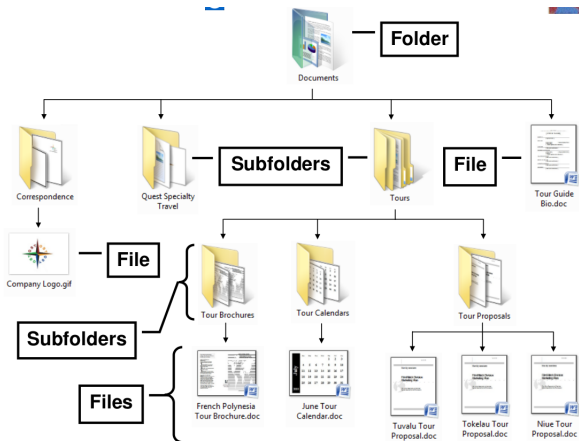
Introduction to UNIX-like systems

Intro. to Computing

`https://mandarm.github.io/computing.html`

File system structure

Files are organised in a hierarchical structure of folders, sub-folders, and files.



Courtesy: <https://www.slideshare.net/okmomwalking/windows-7-unit-b-ppt>

File system structure: terminology

- Folders $\equiv$ *directories*
- Top of the hierarchy: *root directory* (/)
- Default starting location: *home directory* (~)
- Location of a file or directory: specified by *path*
- Current location in terminal or file browser: *current working directory*
- Paths: *absolute* or *relative*
 - absolute path: from root (starts with /)
Example: `/usr/bin/python3`, `/tmp`, `/home/student`
 - relative path: from current working directory (does not start with /)
Example: `Desktop/wsl`

Note the difference between (forward) slash (/, used in Unix-like systems) and backslash (\, used in Windows-like systems) !

Navigating the file system

- **cd** : change directory

Examples:

cd ← go to home directory

cd /home/student (your home directory, same as above)

cd /mnt/c/Users/

- **pwd** : print current working directory

Navigating the file system: special directory names

- `~` : home directory

Example: `cd ~/Desktop`

- `.` (dot) : current working directory

Example: `./program1`

- `..` (dot dot) : parent directory (one level up)

Example: `cd ..`, `cd ../assignment2`, `cd ../..`

- `-` (dash) : go back to the immediately previous location

Example:

`cd /home/student`

`cd /mnt/c/Users`

`cd CSSC`

use your own username instead of CSSC

`cd -`

check location using `pwd`

`cd -`

check location again using `pwd`

Listing files

- `ls` : view list of files in current directory
- `ls <path>` : view list of files in specified path
- `ls -l` : view detailed list of files
- `ls -lt` : view detailed list of files sorted by modification time
- `ls -ltr` : view detailed list of files sorted by modification time *in reverse order*

Managing files from the terminal

Listing files

- `ls` : view list of files in current directory
- `ls <path>` : view list of files in specified path
- `ls -l` : view detailed list of files
- `ls -lt` : view detailed list of files sorted by modification time
- `ls -ltr` : view detailed list of files sorted by modification time *in reverse order*

Example:

```
$ /bin/ls -l
total 68
drwx----- 2 mandar mandar 4096 Jul 19 00:45 assignments
drwx----- 2 mandar mandar 4096 Jul 22 2016 exams
-rw-r--r-- 1 mandar mandar 13521 Jul 19 00:41 index.html
drwx----- 2 mandar mandar 4096 Jul 19 00:45 lectures
```

Essential commands: directories

- `mkdir` : create a directory

Examples:

```
mkdir clab
```

```
mkdir clab/assignment1
```

Create directories

OR

```
mkdir -p clab/assignment1 clab/assignment2
```

- `rmdir` : remove an (empty) directory

Example: `rmdir assignment2`, `rmdir clab/programs`

Essential commands: files

- **cp** : copy a file

```
cp program1.c program2.c
```

```
cp -i source-file target-file
```

```
cp -i source-file target-directory
```

Essential commands: files

■ **cp** : copy a file

```
cp program1.c program2.c
```

```
cp -i source-file target-file
```

```
cp -i source-file target-directory
```

■ **mv** : rename (move) a file

```
mv program1.c program2.c
```

```
mv -i source-file target-file
```

```
mv -i source-file target-directory
```

Essential commands: files

■ `cp` : copy a file

```
cp program1.c program2.c
cp -i source-file target-file
cp -i source-file target-directory
```

■ `mv` : rename (move) a file

```
mv program1.c program2.c
mv -i source-file target-file
mv -i source-file target-directory
```

`-i` $\equiv$ interactive
(asks for confirmation)

■ `rm` : remove (delete) a file

```
rm program1.c
rm -i file1 file2.c *.bak
rm -r some-directory (remove directory and everything inside it)
rm -rf some-directory (remove directory and everything inside it;
dangerous!)
```

Convenience features

Tab completion: type a part of a command / file name, and hit tab to complete or get a list of suggestions.

Aliases

- **What are they?** → user-defined easy-to-remember names for commands (+ arguments)

Examples:

```
alias c="clear -x"
alias cp="cp -i"
alias l="ls -l"
alias ls="ls -F"
alias lt="ls -lt"
alias ltr="ls -ltr"
alias mv="mv -i"
alias rm="rm -i"
```

- **Where to define them?** → in `~/.bash_aliases`
- **How does one create / edit files?** → `notepad.exe bash_aliases`

(“simple and stupid” method that works only on WSL)